Error codes

Every error response has the same shape: an `error` object with a `code` from the table below, a human-readable `message` that may change between releases, the `requestId` of the failed request and, for validation errors, a `details` object naming the parameter. Branch on the code, never on the message.

```
{
  "error": {
    "code": "parameter_invalid",
    "message": "hours must be an integer between 1 and 48",
    "requestId": "req_7f3a9c2e",
    "details": { "parameter": "hours", "received": "0" }
  }
}
```

Field	Type	Description
`code`	string	One of the codes below. Stable; treat unknown codes as errors.
`message`	string	For humans and logs. Not stable.
`requestId`	string	Quote it to support. Also sent as the `X-Request-Id` header.
`details`	object	Present on `400` errors; names the parameter and the value seen.

All codes

Code	HTTP	Meaning	Retry
`parameter_missing`	400	A required parameter is absent.	No
`parameter_invalid`	400	A parameter has the wrong type or is out of range.	No
`range_too_wide`	400	A history range is too long for its step or starts before the retention.	No
`key_invalid`	401	The `Authorization` header is missing or malformed.	No
`key_revoked`	401	The key was revoked or rotated.	No
`key_scope`	403	The key's project does not cover the station.	No
`plan_required`	403	The endpoint or the horizon needs a higher plan.	No
`station_unknown`	404	No station has that code.	No

Code	HTTP	Meaning	Retry
`not_acceptable`	406	`Accept` names a representation other than JSON.	No
`station_retired`	410	The station stopped reporting permanently.	No
`rate_limited`	429	The window is exhausted.	After `Retry-After`
`upstream_delayed`	503	The station has not reported in time.	With backoff
`maintenance`	503	The API is being upgraded; announced in the release notes.	After `Retry-After`

> **Retrying**
>
> The backoff rule for `rate_limited`, `upstream_delayed` and `maintenance` is in Rate limits. No `4xx` other than `429` is worth retrying without changing the request.

Request errors

A `400` describes a request that can be corrected before it is sent again. The `details` object names the parameter and repeats the value the server saw, so nothing has to be parsed out of the message. Validate ranges before sending: `hours` is 1 to 48, `days` 1 to 7, `limit` 1 to 1,000 for history and 1 to 200 for the directory, and `step` is one of `10m`, `1h` and `1d`.

`range_too_wide` is the one `400` that depends on the plan. It is returned when `from` is earlier than the plan's history retention allows, and when a range at the `10m` step spans more than 31 days. The `details` object carries `earliest`, the first instant the plan can serve, so a client can clamp its range and retry once.

Authorization errors

A `401` means the key itself was not accepted. `key_invalid` is almost always a formatting problem: a missing `Bearer` prefix, a key pasted with a trailing newline from a secrets file, or a test key sent to a live station. `key_revoked` follows a rotation or a revocation and is permanent for that key; deploy the new key everywhere before revoking the old one.

A `403` means the key is fine but the request is not. `key_scope` says the station is not in the key's project; add it in the console rather than creating a second project. `plan_required` says the endpoint or the horizon is not in the plan, and its `details.plan` names the lowest plan that allows it.

Availability errors

A `503` is temporary by definition. `upstream_delayed` means the station has been silent for more than six hours; the station directory shows the same in `lastObservedAt`, and the station usually returns within a day. `maintenance` is rare and announced in the release notes beforehand, and it always carries `Retry-After`.

A client that sorts codes into three groups covers every case:

```
switch (body.error?.code) {
  case "rate_limited":
  case "upstream_delayed":
  case "maintenance":
    return retryLater(response)
  case "station_retired":
    return removeStation(station)
  case "key_revoked":
    return alertOperators("Northlight key revoked")
  default:
    throw new Error(`${body.error.code}: ${body.error.message}`)
}
```

> **Exercise every branch**
>
> The synthetic station `TEST-01` accepts a `simulate` parameter naming any code in the table, so every branch of your error handling can be tested without waiting for the real thing.